



Inspiring Excellence

**CSE370 : Database Systems
Project Report**

Project Title : “Things We Show and Tell”

Group No : 01, CSE370 Lab Section : 11, Summer 2026		
ID	Name	Contribution
24301051	Mohammad Arafat	Profiles, Merchant, Transactions
24101165	Ahnaf Ashique Adi	Combat, Tutorials, Dialogues
23301435	Mahdin Samiul Haque	Sessions, Registry, Accounts

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	3-5
3	ER/EER Diagram	6
4	Schema Diagram	7
5	Normalization	8
6	Frontend Development	9-15
7	Backend Development	16-21
8	Source Code Repository	21
9	Conclusion	21
10	References	21

Introduction

The following is an endless roguelike strategy based card game designed around turn-based tactical combat, dialogues, and continuous player tracking. The cards are classified into monsters, which are used for combat and spells, which are used to create dynamic events and change stats and gameplay. Players start session-based runs, earn crayons by combat, purchase cards from a list of randomized cards for better combat, and upgrade their active deck at the merchant. The game also manages current session stats, enforces correct state for real time combat grids across 2 slots and also records high score achievements across player accounts.

Project Features

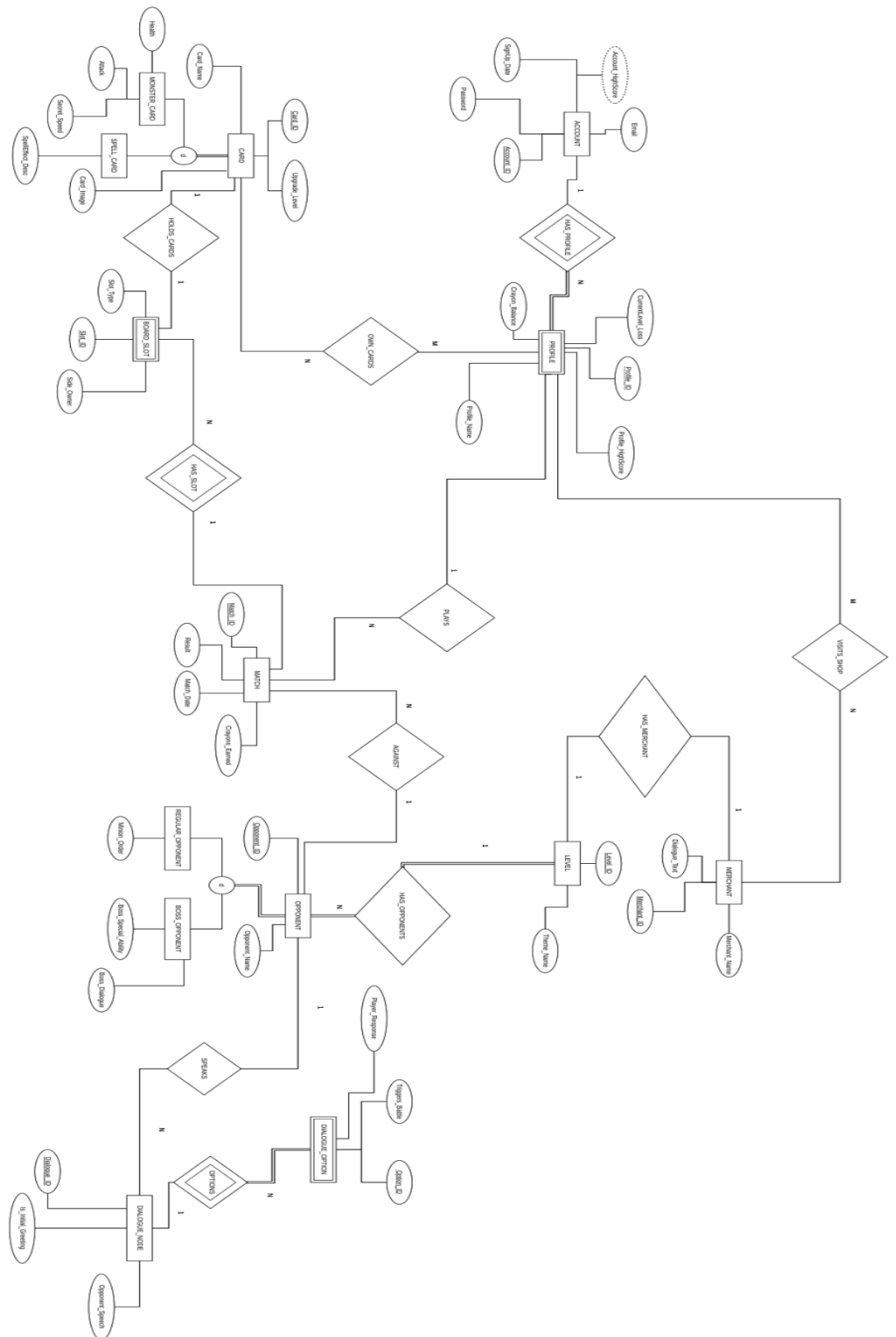
ID, Name	Features	
24301051, Mohammad Arafat	Ft 1	Player Profile Tracking
	Ft 2	Merchant Shop & Transaction
24101165, Ahnaf Ashique Adi	Ft 1	Combat and Slot Tracking
	Ft 2	Tutorial and Dialogues
23301435, Mahdin Samiul Hoque	Ft 1	Game Session Tracking
	Ft 2	Card Template Registry

Member 1	
No.	Feature Description
1	Player Profile Tracking: Manages user accounts and records high scores for individual runs. When a new user registers, their credentials are saved as a new entry in the “Players” entity. A single logged in account can have multiple profiles which are multiple runs, with different state tracking for each of them with individual high scores.
2	Merchant Shop and Transaction: Handles dynamic markets generated between certain stages. The backend populated a temporary table with randomly selected cards, new and old ones which are now upgraded and are linked with the current run. Upon purchase, the database is updated and baseline stats are adjusted as required along with duplicate purchase avoidance and inserting cards in the player ownership.

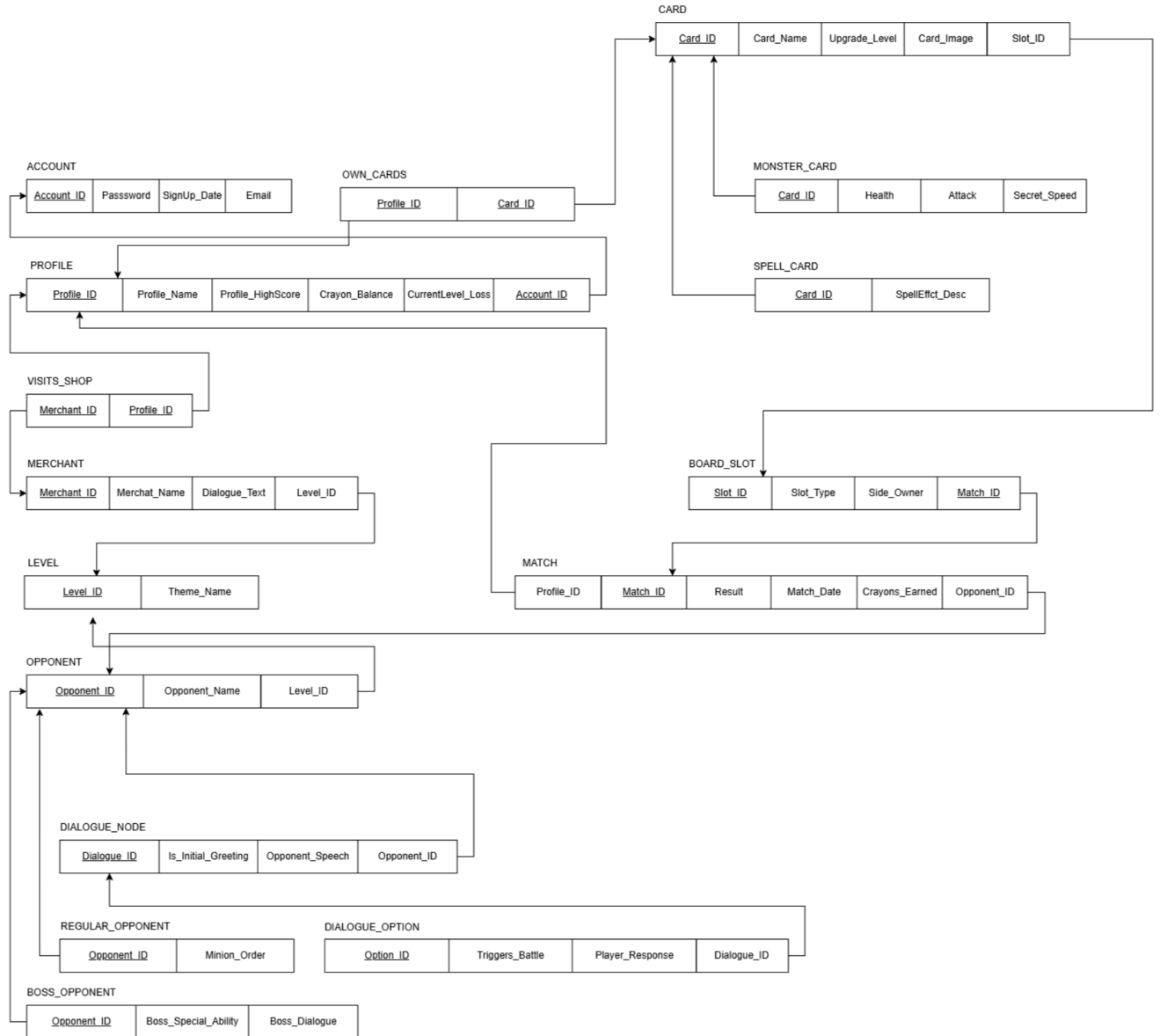
Member 2	
No.	Feature Description
3	Combat and Slot Tracking: Maintains battlefield layout and real-time card stats during active battles. The battlefield is a grid of 4 separate slots (2 for each side with 1 monster zone and 1 spell zone). Dynamic attributes are actively tracked and it is also ensured that all effects and damage during mid-game are retained maintaining database integrity.
4	Tutorial and Dialogues: Manages in-game tutorial instructions and character interaction sequences. All dialogue data is stored directly in the database. The backend uses PHP to query and extract the relevant text based on game progression, delivering it to the frontend to render for the player.

Member 3	
No	Feature Description
5	Game Session Tracking: Manages active play sessions as players progress through the endless matches. Creating a new run creates a new record in the “Matches” table with baseline parameters like, a 100 gold, 0 score, Level 1 and a boolean value to track game state. All subsequent gameplay afterwards including combat, upgrades, purchases, inventory, are linked to the session until player health drops to zero which triggers the ending of a run and sets the required parameters accordingly and also changes the boolean to false.
6	Card Template Registry: The storage for all static cards in the game so that we can prevent redundancy. All cards will be stored generally along with their baseline attributes as a “Card” entity. Moreover, using EER, we can distinguish monster cards and spell cards with their specific traits. Then we can manage player inventory of owned cards and upgrades and fetch them from this place as needed.

ER/EER Diagram



Schema Diagram



Normalization

- a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
- b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
- c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

Answer:

- a) The schema we have produced is in 1NF
- b) The schema we have produced is in 2NF. There are no partial dependencies (all non prime attributes are fully functionally dependent on the primary key) in our schema as if there were, 2NF would not be achieved and we would have had to remove the dependencies to make the schema to 2NF.
- c) The schema we have produced is in 3NF. There are no transitive dependencies in our schema as if there were any, we would have had to convert it to 3NF.

Frontend Development

The frontend was built with HTML, CSS, and JavaScript to provide an interactive and user-friendly game interface. It includes the login/signup screen, profile selection, tutorial pages, inventory view, merchant section, and battle screen. User actions are connected to backend PHP files, and the page updates dynamically to show current cards, scores, balance, and gameplay status in real time.

Contribution of ID : 24301051, Name : Mohammad Arafat (Profile Tracking and Merchant)

Contribution of ID : 24101165, Name : Ahnaf Ashique Adi (Tutorial, Dialogue & Combat)

Contribution of ID : 23301435, Name : Mahdin Samiul Haque (Session Tracking and Card Template Registry)

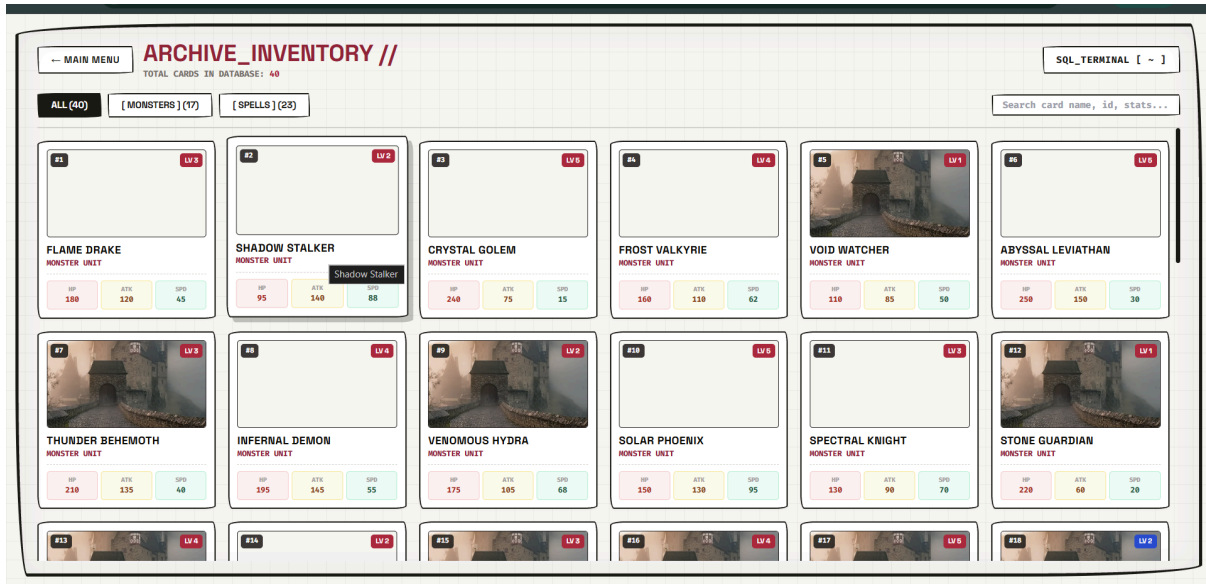
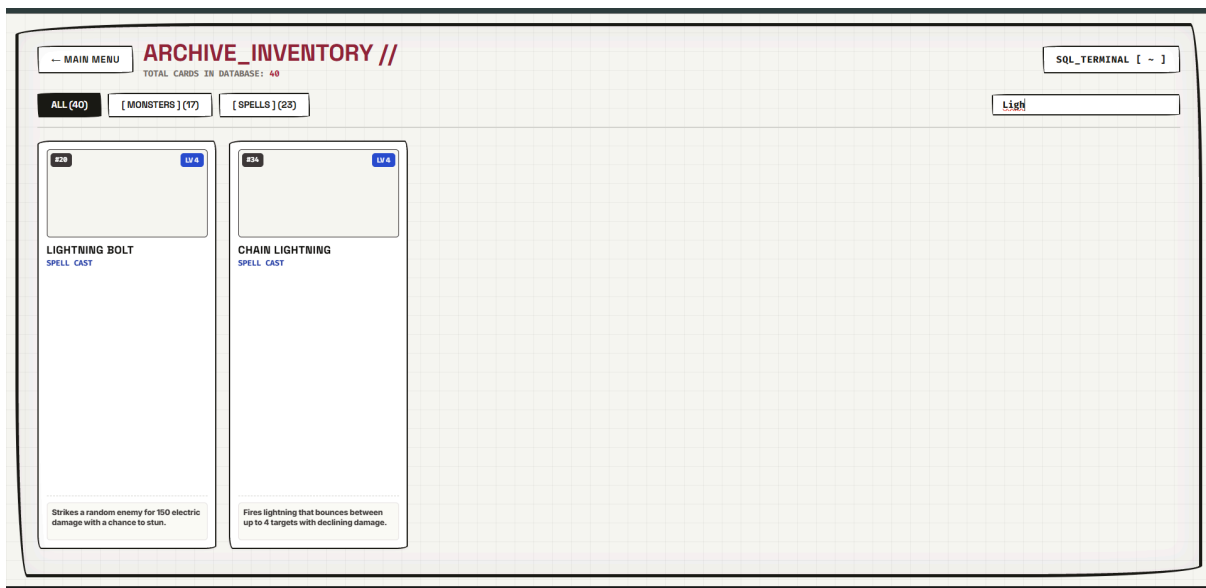
Login and Sign-up

The screenshot shows a web interface titled "SIGN_THE_LEDGER // STUDENT DESK CREDENTIALS". At the top left is a button labeled "-- BACK TO MENU". The main content area contains two input fields: "EMAIL ADDRESS" with the placeholder "student@course.edu" and "PASSCODE" with a masked input "*****". Below these fields are two buttons: "LOGIN" and "SIGN UP". A small footnote at the bottom left states: "* Raw SQL queries execute against ACCOUNT table." On the right side of the screen is a large square placeholder for a profile picture, featuring a stylized character with a white face and a black mask.

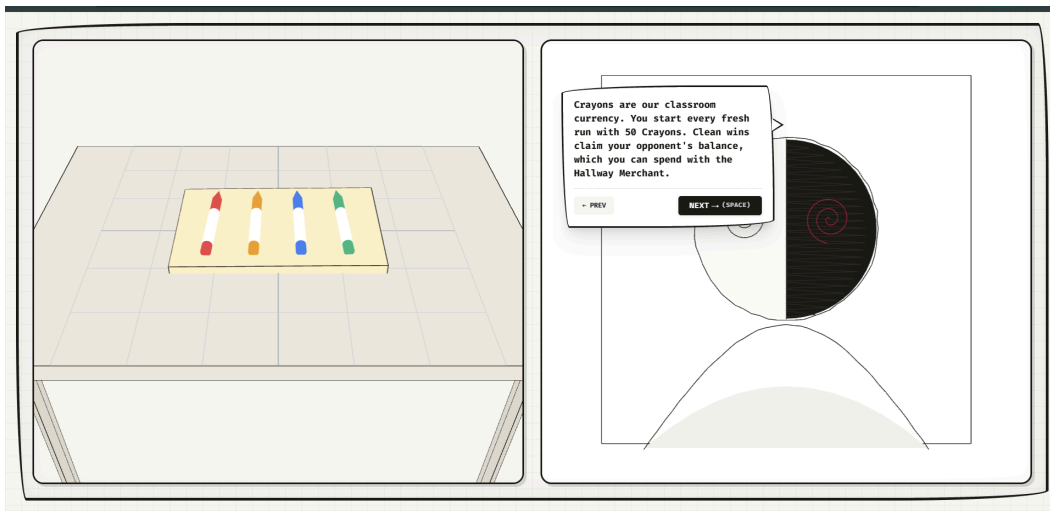
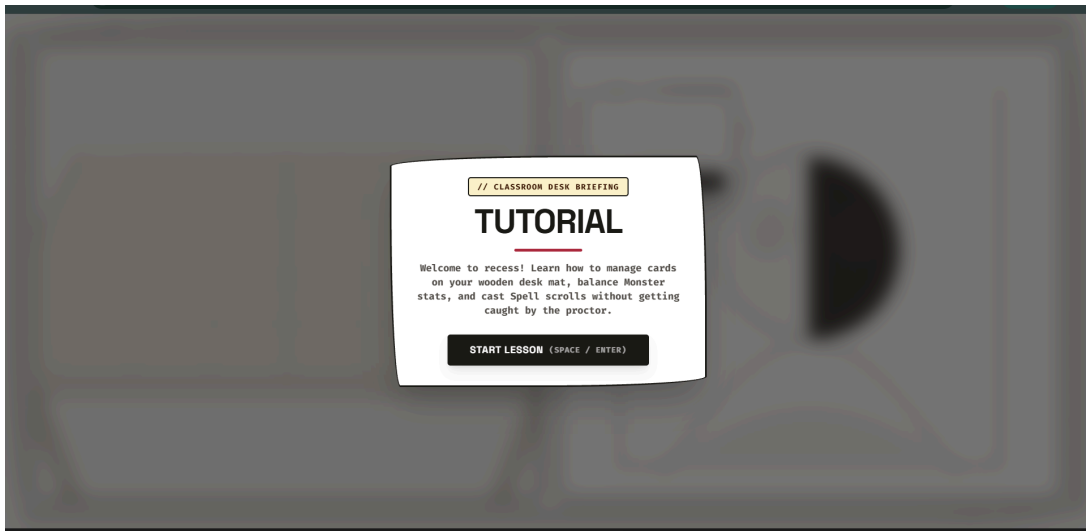
Debug panel

The screenshot shows a web interface titled "DEBUG WORKSPACE // SYSTEM DIAGNOSTICS & DATABASE ATLAS". At the top left is a button labeled "-- BACK TO MENU". The main content area is divided into three sections: "TYPOGRAPHY PALETTE" with a sub-label "Switch font pairings" and three buttons labeled "[TECHNICAL]", "[BLUEPRINT]", and "[EDITORIAL]"; "RAW SQL MONITOR" with a sub-label "Toggle transaction inspector" and a "TOGGLE" button; and "CARD INVENTORY ATLAS" with a sub-label "Open live MySQL cards catalog (40 items)" and a "LAUNCH" button. A small footnote at the bottom left states: "* Press [=] to toggle the SQL monitor anytime." On the right side of the screen is a large square placeholder for a profile picture, featuring a stylized character with a white face and a black mask.

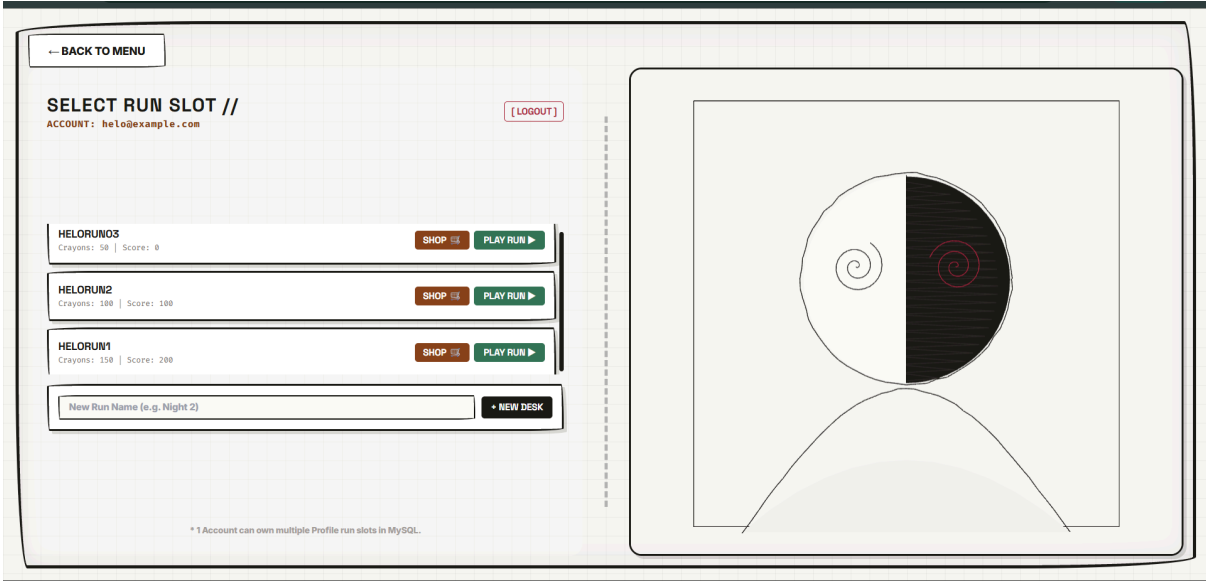
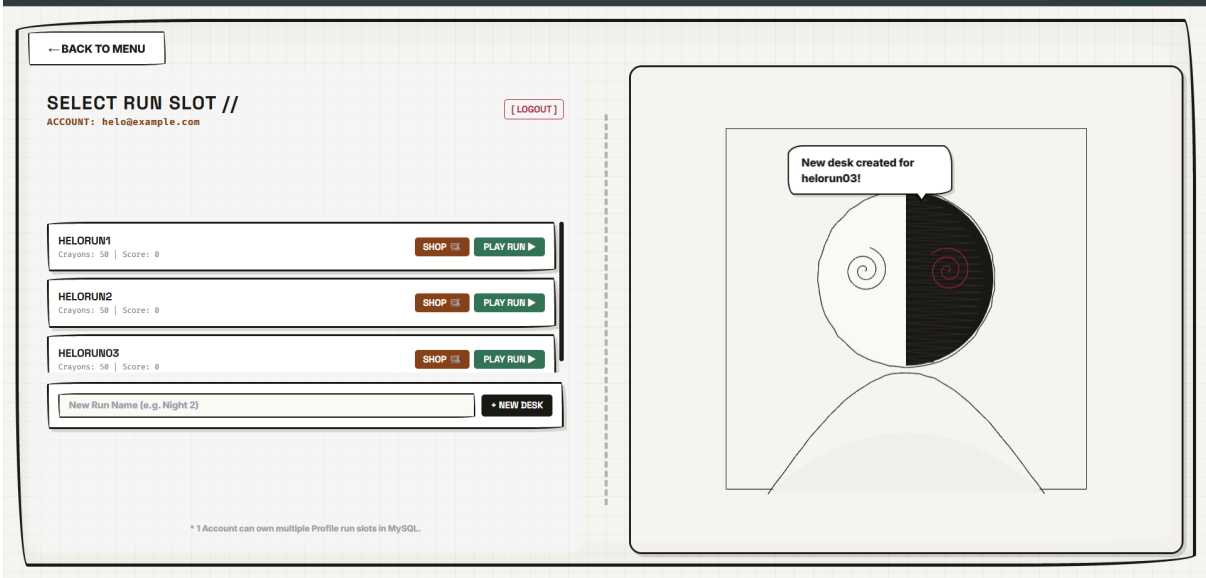
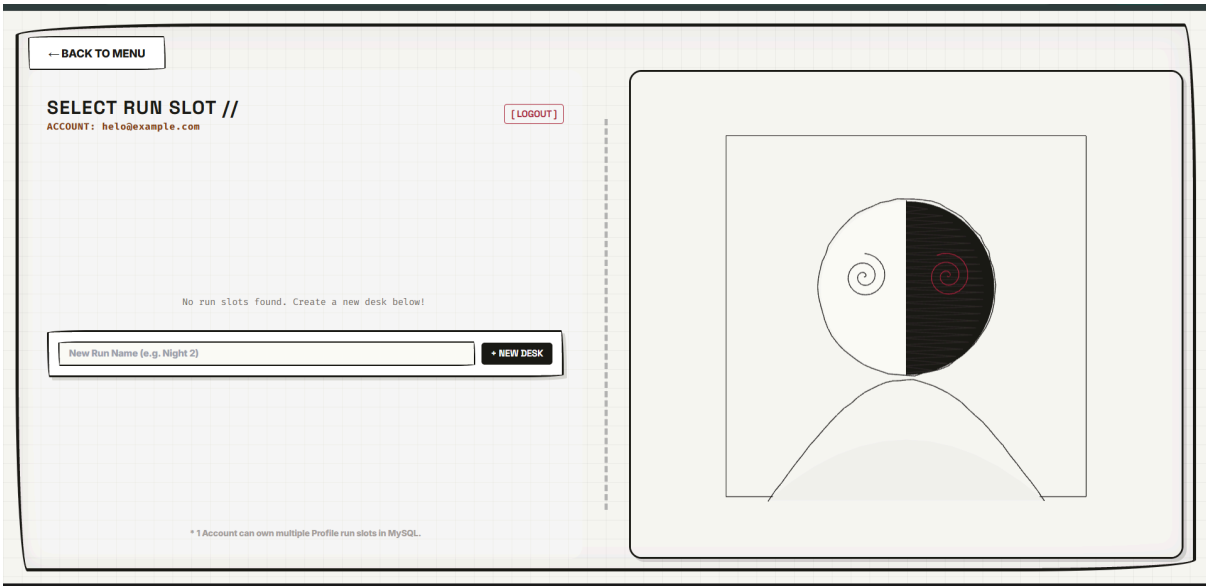
Card template registry:



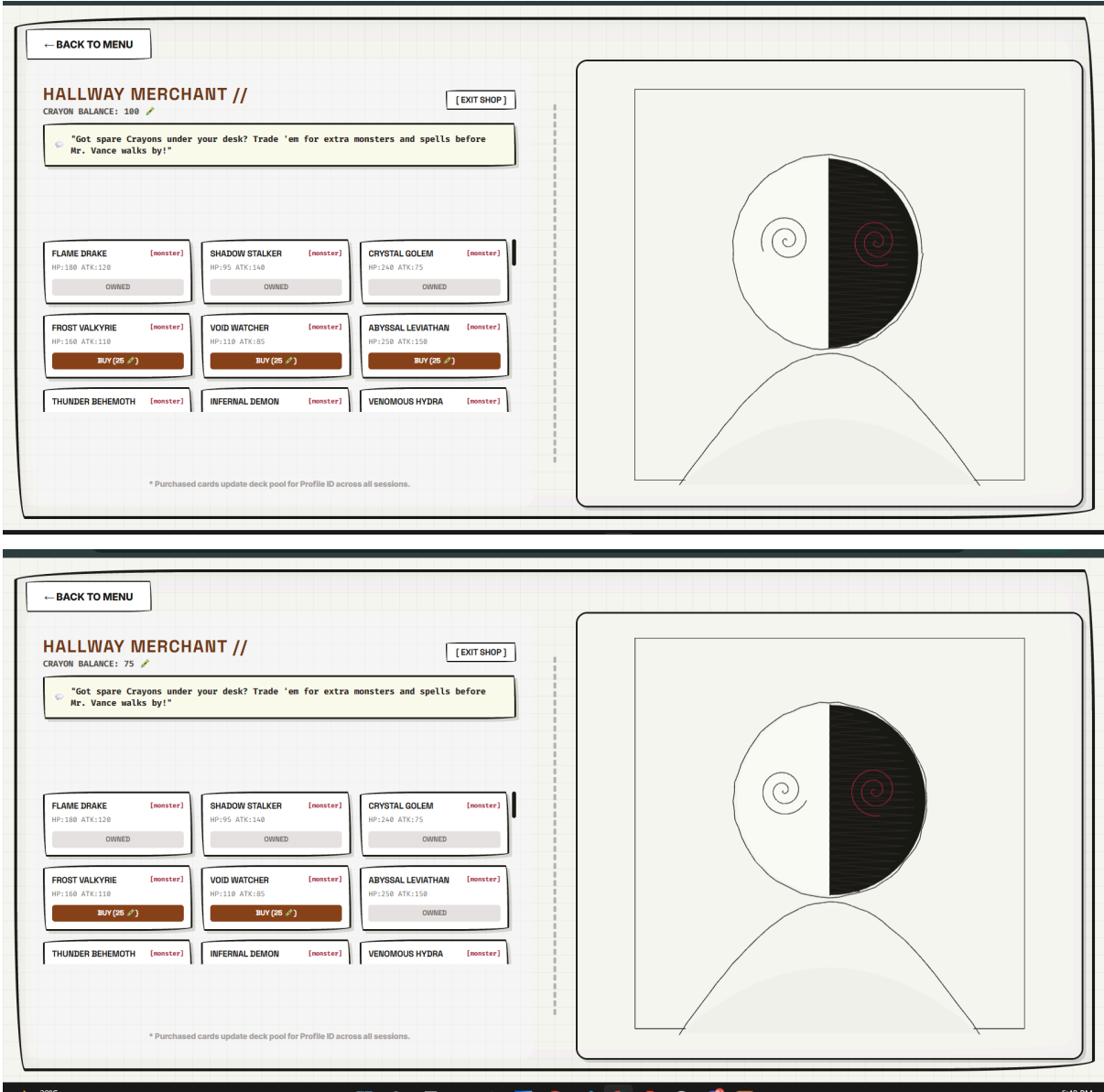
Tutorial and dialogues



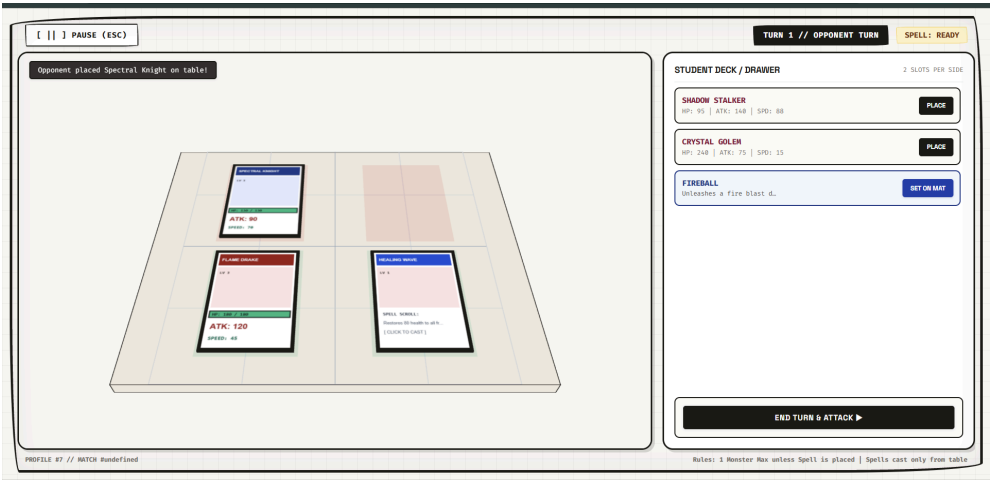
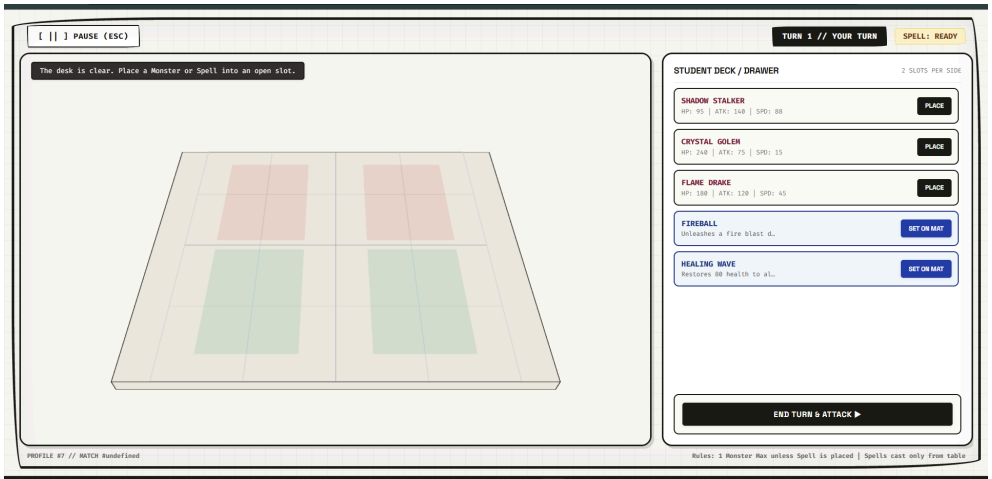
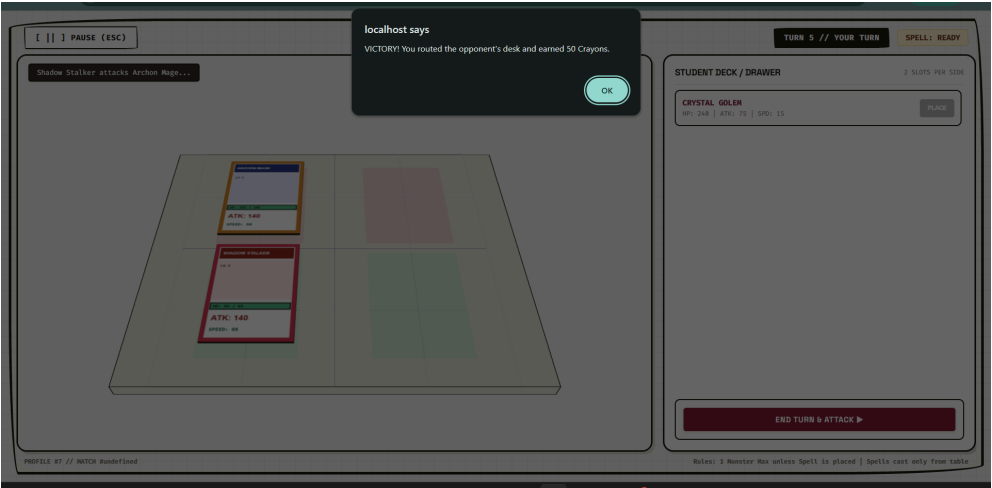
Player Profile Tracking



Merchant shop and Transaction



Combat and Slot Tracking



Game Session Tracking

HELORUN03 Crayons: 100 Score: 100	SHOP	PLAY RUN
HELORUN2 Crayons: 75 Score: 100	SHOP	PLAY RUN
HELORUN1 Crayons: 150 Score: 200	SHOP	PLAY RUN

HELORUN03 Crayons: 50 Score: 0	SHOP	PLAY RUN
HELORUN2 Crayons: 100 Score: 100	SHOP	PLAY RUN
HELORUN1 Crayons: 150 Score: 200	SHOP	PLAY RUN

Backend Development

The backend of this project was developed using PHP and MySQL to manage the game's core logic and database operations. It handles account registration and login, profile/run creation, tutorial progress, card inventory, merchant purchases, and match result updates. SQL queries are used to read and store game data, and the backend returns structured responses so the frontend can keep player balance, score, cards, and progress synchronized.

Contribution of ID : 24301051, Name : Mohammad Arafat

1) Merchant Screen (merchant.php)

```
---> SELECT Profile_ID, Profile_Name, Crayon_Balance, Account_ID  
FROM PROFILE  
WHERE Profile_ID = :pid
```

```
---> SELECT Card_ID FROM OWN_CARDS WHERE Profile_ID = :pid
```

```
---> SELECT c.Card_ID, c.Card_Name, c.Upgrade_Level, c.Card_Image,  
        CASE WHEN m.Card_ID IS NOT NULL THEN 'monster' ELSE 'spell' END AS card_type,  
        m.Health, m.Attack, m.Secret_Speed, s.Spelleffect_Desc  
FROM CARD c  
LEFT JOIN MONSTER_CARD m ON c.Card_ID = m.Card_ID  
LEFT JOIN SPELL_CARD s ON c.Card_ID = s.Card_ID  
ORDER BY c.Card_ID ASC
```

2) Buy Card (buy_card.php)

```
--->SELECT Crayon_Balance FROM PROFILE WHERE Profile_ID = :pid FOR UPDATE
```

```
--->SELECT COUNT(*) FROM OWN_CARDS WHERE Profile_ID = :pid AND Card_ID = :cid
```

```
--->UPDATE PROFILE  
SET Crayon_Balance = Crayon_Balance - :price  
WHERE Profile_ID = :pid
```

```
--->INSERT INTO OWN_CARDS (Profile_ID, Card_ID) VALUES (:pid, :cid)
```


Contribution of ID : 24101165, Name : Ahnaf Ashique Adi

1) Initialize Match (init_match.php)

```
--->SELECT c.Card_ID, c.Card_Name, c.Upgrade_Level, c.Card_Image,  
        CASE WHEN m.Card_ID IS NOT NULL THEN 'monster' ELSE 'spell' END AS card_type,  
        m.Health, m.Attack, m.Secret_Speed, s.Spelleffect_Desc  
FROM OWN_CARDS oc  
JOIN CARD c ON oc.Card_ID = c.Card_ID  
LEFT JOIN MONSTER_CARD m ON c.Card_ID = m.Card_ID  
LEFT JOIN SPELL_CARD s ON c.Card_ID = s.Card_ID  
WHERE oc.Profile_ID = :pid
```

```
--->SELECT c.Card_ID, c.Card_Name, c.Upgrade_Level, c.Card_Image, 'monster' AS card_type,  
        m.Health, m.Attack, m.Secret_Speed  
FROM CARD c  
JOIN MONSTER_CARD m ON c.Card_ID = m.Card_ID  
LIMIT 3
```

```
--->SELECT c.Card_ID, c.Card_Name, c.Upgrade_Level, c.Card_Image, 'spell' AS card_type,  
        s.Spelleffect_Desc  
FROM CARD c  
JOIN SPELL_CARD s ON c.Card_ID = s.Card_ID  
LIMIT 2
```

```
--->INSERT IGNORE INTO OWN_CARDS (Profile_ID, Card_ID) VALUES (:pid, :cid)
```

```
--->SELECT c.Card_ID, c.Card_Name, c.Upgrade_Level, c.Card_Image,  
        'monster' AS card_type, m.Health, m.Attack, m.Secret_Speed  
FROM CARD c  
JOIN MONSTER_CARD m ON c.Card_ID = m.Card_ID  
ORDER BY RAND() LIMIT 3
```

```
--->SELECT c.Card_ID, c.Card_Name, c.Upgrade_Level, c.Card_Image,  
        'spell' AS card_type, s.Spelleffect_Desc  
FROM CARD c  
JOIN SPELL_CARD s ON c.Card_ID = s.Card_ID  
ORDER BY RAND() LIMIT 2
```

2) Update Board Slot (update_board_slot.php)

```
--->SELECT Slot_ID FROM BOARD_SLOT  
WHERE Match_ID = :mid AND Side_Owner = :owner AND Slot_Type = :slot_type  
LIMIT 1
```

```
--->UPDATE CARD SET Slot_ID = :slot_id WHERE Card_ID = :card_id
```

```
--->UPDATE CARD SET Slot_ID = NULL WHERE Slot_ID = :slot_id
```

3) End Match (end_match.php)

```
--->UPDATE `MATCH`  
SET Result = :result, Crayons_Earned = :earned  
WHERE Match_ID = :mid
```

```
--->UPDATE PROFILE  
SET Crayon_Balance = Crayon_Balance + :earned,  
  Profile_HighScore = Profile_HighScore + 100  
WHERE Profile_ID = :pid
```

```
--->SELECT Crayon_Balance, Profile_HighScore FROM PROFILE WHERE Profile_ID = :pid
```

4) Get Dialogues (get_dialogues.php)

```
--->SELECT Dialogue_ID, Is_Initial_Greeting, Opponent_Speech, Opponent_ID  
FROM dialogue_node  
ORDER BY Dialogue_ID ASC
```

5) Tutorial Cards (get_tutorial_cards.php)

```
--->SELECT c.Card_ID, c.Card_Name, c.Card_Image, c.Upgrade_Level, 'MONSTER' AS card_type,  
  m.Health, m.Attack, m.Secret_Speed  
FROM CARD c  
INNER JOIN MONSTER_CARD m ON c.Card_ID = m.Card_ID  
ORDER BY c.Card_ID ASC  
LIMIT 2
```

```
--->SELECT c.Card_ID, c.Card_Name, c.Card_Image, c.Upgrade_Level, 'SPELL' AS card_type,  
  s.SpellEffect_Desc  
FROM CARD c  
INNER JOIN SPELL_CARD s ON c.Card_ID = s.Card_ID  
ORDER BY c.Card_ID ASC  
LIMIT 1
```

6) Complete Tutorial (complete_tutorial.php)

```
--->UPDATE PROFILE  
SET Crayon_Balance = GREATEST(COALESCE(Crayon_Balance, 0), 50)  
WHERE Profile_ID = :profile_id
```

Contribution of ID : 23301435, Name : Mahdin Samiul Haque

1) Sign Up (signup.php)

```
---> INSERT INTO ACCOUNT (Email, Password) VALUES (:email, :password)
```

2) Login (login.php)

```
---> SELECT Account_ID, Password, Email FROM ACCOUNT WHERE Email = :email
```

```
---> SELECT Profile_ID, Profile_Name, Profile_HighScore, Crayon_Balance, CurrentLevel_Loss  
FROM PROFILE  
WHERE Account_ID = :account_id
```

3) Create Profile / New Run Slot (create_profile.php)

```
---> INSERT INTO PROFILE (Profile_Name, Account_ID) VALUES (:profile_name, :account_id)
```

4) Get Account Runs (get_account_runs.php)

```
---> SELECT Account_ID, Email FROM ACCOUNT WHERE Account_ID = :account_id LIMIT 1
```

```
---> SELECT Profile_ID, Profile_Name, Crayon_Balance, Profile_HighScore  
FROM PROFILE  
WHERE Account_ID = :account_id  
ORDER BY Profile_ID DESC
```

5) Inventory Cards (get_cards.php)

```
---> SELECT  
    c.Card_ID,  
    c.Card_Name,  
    c.Upgrade_Level,  
    c.Card_Image,  
    c.Slot_ID,
```

```

m.Health,
m.Attack,
m.Secret_Speed,
s.SpellEffect_Desc,
CASE
    WHEN m.Card_ID IS NOT NULL THEN 'monster'
    WHEN s.Card_ID IS NOT NULL THEN 'spell'
    ELSE 'unknown'
END AS card_type
FROM CARD c
LEFT JOIN MONSTER_CARD m ON c.Card_ID = m.Card_ID
LEFT JOIN SPELL_CARD s ON c.Card_ID = s.Card_ID
ORDER BY c.Card_ID ASC

```

Source Code Repository

[Ahnaf-Ashique-Adi/CSE370-LAB-PROJECT-](#)

Conclusion

In this project we successfully developed a complete card-based game system by combining frontend design with backend database logic. The system supports user account management, profile-based game runs, tutorial progression, card collection, merchant purchases, and match result tracking in a connected workflow. Overall, the project demonstrates practical use of HTML, CSS, JavaScript, PHP, and MySQL in building a functional and interactive software application with proper teamwork and feature integration.

References

- 1.MDN Web Docs – HTML, CSS, JavaScript
<https://developer.mozilla.org/>
- 2.PHP Manual (Official Documentation)
<https://www.php.net/docs.php>
- 3.MySQL 8.0 Reference Manual
<https://dev.mysql.com/doc/>
- 4.W3Schools Web Development Tutorials (for basic syntax support)
<https://www.w3schools.com/>
- 5.Course materials and lab instructions provided in CSE370.